

Servidor HTTP/1.1 sobre Sockets TCP

Laboratório de Redes de Computadores — Trabalho 1

Anthony Antonelli Leonardo Fagundes Lucas Simão Samuel Bitdinger

Testes realizados em 3 de outubro de 2026

1 Arquitetura e concorrência

O servidor foi escrito em Java e usa apenas `java.net.ServerSocket` e `java.net.Socket`. Nenhuma biblioteca HTTP de servidor é utilizada: a leitura da requisição e a montagem da resposta são feitas pelo nosso código. As responsabilidades estão divididas em seis classes:

- `ServerConfig` lê e valida os argumentos `--port`, `--root`, `--idle-timeout` e `--workers`.
- `ServerMain` abre o socket em `0.0.0.0`, aceita conexões e entrega cada uma a um *pool* fixo de threads.
- `HttpRequestReader` acumula os bytes recebidos em um buffer até encontrar `\r\n\r\n`. O que vier depois desse marcador fica guardado para a requisição seguinte, já que o TCP entrega um fluxo de bytes e uma leitura pode trazer meia requisição ou uma requisição e o começo da próxima.
- `StaticFileService` decodifica o percent-encoding, normaliza o caminho e só abre o arquivo se o caminho real estiver dentro da raiz.
- `HttpServletResponse` escreve a linha de status e os cabeçalhos `Date` (IMF-fixdate em GMT), `Server`, `Content-Type` e `Content-Length`. No GET o corpo sai em blocos de até 16 KiB; no HEAD os cabeçalhos são os mesmos e o corpo não é enviado.
- `HttpConnectionHandler` atende requisições em sequência na mesma conexão até receber `Connection: close`, estourar o timeout de ociosidade ou o cliente fechar o socket.

Estratégia de concorrência. Adotamos uma thread por conexão, tiradas de um *pool* de tamanho fixo (padrão: o maior entre 4 e o número de CPUs). O modelo é bloqueante, então o código de cada conexão é linear e fácil de explicar: lê, processa, responde, repete. O *pool* limita o número de threads, o que impede que uma rajada de conexões crie threads sem controle. Uma requisição lenta ocupa só a própria thread e as outras conexões seguem sendo atendidas enquanto houver thread livre. Quando todas estão ocupadas, as novas conexões esperam na fila do executor.

Duas limitações conhecidas: a fila do executor não tem tamanho máximo, e o timeout vale para leitura, não para escrita. Um cliente que pare de ler a resposta pode segurar uma thread.

Configuração usada nos testes. Porta 8080, raiz `./www`, timeout de ociosidade de 5000 ms. O servidor rodou em um MacBook (macOS) e o cliente principal em uma máquina Windows da mesma rede local, com `curl.exe` no PowerShell. Os endereços IP foram omitidos do texto e aparecem borrados nas figuras.

2 Conformidade HTTP

A Tabela 1 traz, para cada código obrigatório, o comando executado no cliente e a resposta recebida. Em todos os casos a base é `http://<servidor>:8080`. As Figuras 1 a 2 mostram as saídas obtidas.

Código	Requisição	Resposta obtida
200 (GET)	<code>curl.exe --http1.1 -i "\$BASE/index.html"</code>	HTTP/1.1 200 OK, Content-Length: 2294, Content-Type: text/html; charset=utf-8, corpo com o HTML
200 (HEAD)	<code>curl.exe --http1.1 -I "\$BASE/index.html"</code>	HTTP/1.1 200 OK, mesmos cabeçalhos do GET (Content-Length: 2294), sem corpo
400	<code>curl.exe --http1.1 -i --request-target '/ alvo-invalido' "\$BASE/"</code>	HTTP/1.1 400 Bad Request, Content-Length: 17, Connection: close
403	<code>curl.exe --http1.1 -i --path-as-is "\$BASE/../../etc/passwd"</code>	HTTP/1.1 403 Forbidden, Content-Length: 15 (ver Seção 3)
404	<code>curl.exe --http1.1 -i "\$BASE/aaa.html"</code>	HTTP/1.1 404 Not Found, Content-Length: 15
405	<code>curl.exe --http1.1 -i -X POST "\$BASE/index.html"</code>	HTTP/1.1 405 Method Not Allowed, Content-Length: 24, Allow: GET, HEAD

Tabela 1: Códigos de status obrigatórios, requisição e resposta.

Todas as respostas trazem **Date** em GMT e o cabeçalho **Server** com o valor **Grupo-Redes-HTTP1.1**. O **Content-Length** de 2294 bytes é o tamanho exato de `www/index.html`, e o valor é o mesmo no GET e no HEAD. As respostas de erro têm **Content-Length** igual ao tamanho da mensagem curta enviada no corpo. O 400 fecha a conexão porque, depois de uma linha de requisição inválida, o servidor não tem como saber onde começa a próxima.

```
PS C:\Users\samue\Documents\Projetos\t1 redes> curl.exe --http1.1 -i "http://:8080/index.html"
HTTP/1.1 200 OK
Date: Sat, 03 Oct 2026 21:12:29 GMT
Server: Grupo-Redes-HTTP1.1
Content-Length: 2294
Content-Type: text/html; charset=utf-8

<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Servidor HTTP/1.1 - Redes de Computadores</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
```

Figura 1: GET de `index.html`: 200 OK com os cabeçalhos e o começo do corpo.

```
PS C:\Users\samue\Documents\Projetos\t1 redes> curl.exe --http1.1 -I "http://[redacted]:8080/index.html"
HTTP/1.1 200 OK
Date: Sat, 03 Oct 2026 21:13:58 GMT
Server: Grupo-Redes-HTTP1.1
Content-Length: 2294
Content-Type: text/html; charset=utf-8

PS C:\Users\samue\Documents\Projetos\t1 redes> curl.exe --http1.1 -i --request-target '/' alvo-invalidado' "http://[redacted]:8080/"
HTTP/1.1 400 Bad Request
Date: Sat, 03 Oct 2026 21:15:22 GMT
Server: Grupo-Redes-HTTP1.1
Content-Length: 17
Content-Type: text/plain; charset=utf-8
Connection: close
400 Bad Request

PS C:\Users\samue\Documents\Projetos\t1 redes> curl.exe --http1.1 -i "http://[redacted]:8080/aaa.html"
HTTP/1.1 404 Not Found
Date: Sat, 03 Oct 2026 21:17:19 GMT
Server: Grupo-Redes-HTTP1.1
Content-Length: 15
Content-Type: text/plain; charset=utf-8
404 Not Found

PS C:\Users\samue\Documents\Projetos\t1 redes> curl.exe --http1.1 -i -X POST "http://[redacted]:8080/index.html"
HTTP/1.1 405 Method Not Allowed
Date: Sat, 03 Oct 2026 21:18:06 GMT
Server: Grupo-Redes-HTTP1.1
Content-Length: 24
Content-Type: text/plain; charset=utf-8
Allow: GET, HEAD
405 Method Not Allowed
```

Figura 2: De cima para baixo: HEAD (200, sem corpo), linha de requisição inválida (400), arquivo inexistente (404) e POST (405 com Allow).

3 Segurança: travessia de diretório

O StaticFileService decodifica o caminho, resolve os .. e compara o caminho real resultante com a raiz configurada. Se o resultado sair da raiz, a resposta é 403, exista o arquivo ou não. A opção --path-as-is é necessária porque, sem ela, o próprio curl remove os ../ antes de enviar.

Tentativa	Caminho enviado	Resposta
../ literal	/../../../../etc/passwd	HTTP/1.1 403 Forbidden
../ literal	/../../../../outside.txt	HTTP/1.1 403 Forbidden
Percent-encoding	/%2e%2e/%2e%2e/outside.txt	HTTP/1.1 403 Forbidden
Prefixo válido + encoding	/safe/%2e%2e/%2e%2e/outside.txt	HTTP/1.1 403 Forbidden

Tabela 2: Tentativas de travessia rejeitadas pelo servidor.

```

C:\Users\...>curl.exe --http1.1 -i --path-as-is "$BASE/safe/%2e%2e/%2e%2e/outside.txt"curl.exe --http1.1 -i --path-as-is "http://192.168.1.100:8080/.../e
tc/passwd"curl.exe --http1.1 -i --path-as-is "http://192.168.1.100:8080/.../outside.txt"curl.exe --http1.1 -i --path-as-is "http://192.168.1.100:8080/%2e%2e/%2e%2e/ou
tside.txt"curl.exe --http1.1 -i --path-as-is "http://192.168.1.100:8080/safe/%2e%2e/%2e%2e/outside.txt"
curl: (3) URL rejected: Bad hostname
HTTP/1.1 403 Forbidden
Date: Mon, 05 Oct 2026 16:09:07 GMT
Server: Grupo-Redes-HTTP1.1
Content-Length: 15
Content-Type: text/plain; charset=utf-8

403 Forbidden
HTTP/1.1 403 Forbidden
Date: Mon, 05 Oct 2026 16:09:07 GMT
Server: Grupo-Redes-HTTP1.1
Content-Length: 15
Content-Type: text/plain; charset=utf-8

403 Forbidden
HTTP/1.1 403 Forbidden
Date: Mon, 05 Oct 2026 16:09:08 GMT
Server: Grupo-Redes-HTTP1.1
Content-Length: 15
Content-Type: text/plain; charset=utf-8

403 Forbidden
HTTP/1.1 403 Forbidden
Date: Mon, 05 Oct 2026 16:09:08 GMT
Server: Grupo-Redes-HTTP1.1
Content-Length: 15
Content-Type: text/plain; charset=utf-8

403 Forbidden

```

Figura 3: As quatro tentativas, feitas a partir da máquina cliente. Com `-i`, o curl mostra a linha de status e os cabeçalhos: todas recebem HTTP/1.1 403 Forbidden.

4 Transação GET completa

A transação analisada é a primeira conexão da captura `capturas/c1.pcapng`, feita no servidor com o filtro `tcp.port == 8080`. O cliente Windows executou `curl.exe --http1.1 -H 'Connection: close' "$BASE/index.html"`; a Figura 4 mostra a mesma requisição vista pelo terminal. A Figura 5 mostra a conexão no Wireshark, isolada com o filtro `tcp.stream eq 0`, e a Tabela 3 detalha cada quadro, com o tempo contado a partir do SYN.

```

PS C:\Users\samuel\Documents\Projetos\t1 redes> curl.exe --http1.1 -i -H 'Connection: close' "http://192.168.1.100:8080/index.html"
HTTP/1.1 200 OK
Date: Sat, 03 Oct 2026 21:19:13 GMT
Server: Grupo-Redes-HTTP1.1
Content-Length: 2294
Content-Type: text/html; charset=utf-8
Connection: close

```

Figura 4: GET com `Connection: close`, visto pelo cliente (início da saída).

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000			TCP	66	50479 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
2	0.000351			TCP	66	8080 → 50479 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 SACK_PERM
3	0.003717			TCP	60	50479 → 8080 [ACK] Seq=1 Ack=1 Win=65280 Len=0
4	0.003719			HTTP	167	GET /index.html HTTP/1.1
5	0.003808			TCP	54	8080 → 50479 [ACK] Seq=1 Ack=114 Win=262080 Len=0
6	0.006030			TCP	220	8080 → 50479 [PSH, ACK] Seq=1 Ack=114 Win=262144 Len=166 [TCP PDU reassembled in 8]
7	0.006159			TCP	1514	8080 → 50479 [ACK] Seq=167 Ack=114 Win=262144 Len=1460 [TCP PDU reassembled in 8]
8	0.006166			HTTP	888	HTTP/1.1 200 OK (text/html)
9	0.006272			TCP	54	8080 → 50479 [FIN, ACK] Seq=2461 Ack=114 Win=262144 Len=0
10	0.010026			TCP	60	50479 → 8080 [ACK] Seq=114 Ack=2462 Win=65280 Len=0
11	0.010027			TCP	60	50479 → 8080 [FIN, ACK] Seq=114 Ack=2462 Win=65280 Len=0
13	0.010133			TCP	54	8080 → 50479 [ACK] Seq=2462 Ack=115 Win=262144 Len=0

Figura 5: Primeira conexão de `c1.pcapng` no Wireshark: handshake (1 a 3), GET (4), confirmação e resposta 200 OK (5 a 8) e encerramento (9 a 13). As colunas de endereço IP foram borradas.

Quadro	t (ms)	Sentido	Flags	Bytes	Conteúdo
1	0,000	cliente → servidor	SYN	66	abertura
2	0,351	servidor → cliente	SYN, ACK	66	abertura
3	3,717	cliente → servidor	ACK	60	fim do handshake
4	3,719	cliente → servidor	PSH, ACK	167	GET /index.html HTTP/1.1 (113 bytes)
5	3,808	servidor → cliente	ACK	54	confirma a requisição
6	6,030	servidor → cliente	PSH, ACK	220	HTTP/1.1 200 OK e cabeçalhos (166 bytes)
7	6,159	servidor → cliente	ACK	1514	corpo, 1460 bytes
8	6,166	servidor → cliente	PSH, ACK	888	corpo, 834 bytes
9	6,272	servidor → cliente	FIN, ACK	54	servidor encerra
10	10,026	cliente → servidor	ACK	60	confirma os dados e o FIN
11	10,027	cliente → servidor	FIN, ACK	60	cliente encerra
13	10,133	servidor → cliente	ACK	54	confirma o FIN

Tabela 3: Quadros da primeira conexão de `c1.pcapng`: handshake (1 a 3), requisição e resposta (4 a 8) e encerramento (9 a 13). O quadro 12 é o SYN da conexão seguinte.

O corpo de 2294 bytes não cabe em um segmento (MSS de 1460 bytes) e sai em dois, 1460 + 834. Como a requisição pediu `Connection: close`, é o servidor que manda o primeiro FIN, logo depois do último segmento do corpo.

5 Atendimento simultâneo

O teste está em `capturas/concorrencia.pcapng` e usa duas máquinas clientes, com IPs diferentes. O cliente A abre uma conexão e envia só o começo da requisição (a linha `GET` e o cabeçalho `Host`, sem a linha em branco final). A thread que atende A fica bloqueada esperando o resto. Enquanto isso, o cliente B faz um GET completo.

t (s)	Cliente	Quadros	Evento
0,000	A	1 a 4	handshake
0,395	A	5	requisição parcial, 49 bytes, sem <code>\r\n\r\n</code>
4,921	B	9 a 11	handshake
4,939	B	13	GET /index.html completo
4,967	B	15 a 17	200 OK e corpo (2294 bytes)
4,982	B	19 a 22	encerramento
10,402	A	23	resto da requisição (<code>Connection: close</code> e linha em branco)
10,404	A	25 a 27	200 OK e corpo (2294 bytes)
10,414	A	28 a 32	encerramento

Tabela 4: Linha do tempo de `concorrencia.pcapng`. A conexão de A fica aberta de 0 a 10,4s; B é atendido por inteiro nesse intervalo.

B recebeu a resposta 28 ms depois de enviar o GET, com a conexão de A ainda aberta e incompleta. Um servidor de uma thread só ficaria parado na leitura de A e B esperaria mais de 5 s. Depois, A completou a requisição e também recebeu 200. Para que a requisição parcial de A não expirasse durante os 10s do teste, o servidor foi iniciado com `--idle-timeout` maior que o padrão.

6 RTT entre as máquinas

O RTT foi medido com dez pacotes ICMP entre as duas máquinas usadas nos testes (Figura 6). Não houve perda.

Mínimo	Médio	Máximo	Desvio padrão
2,756 ms	3,918 ms	7,730 ms	1,907 ms

Tabela 5: RTT medido com `ping -c 10 <IP>`.

```
[samuelbitdinger@Samuels-MacBook-Air ~ % ping -c 10   
PING ( ): 56 data bytes  
64 bytes from : icmp_seq=0 ttl=128 time=3.764 ms  
64 bytes from : icmp_seq=1 ttl=128 time=2.756 ms  
64 bytes from : icmp_seq=2 ttl=128 time=7.651 ms  
64 bytes from : icmp_seq=3 ttl=128 time=2.798 ms  
64 bytes from : icmp_seq=4 ttl=128 time=2.872 ms  
64 bytes from : icmp_seq=5 ttl=128 time=7.730 ms  
64 bytes from : icmp_seq=6 ttl=128 time=2.796 ms  
64 bytes from : icmp_seq=7 ttl=128 time=2.897 ms  
64 bytes from : icmp_seq=8 ttl=128 time=2.808 ms  
64 bytes from : icmp_seq=9 ttl=128 time=3.104 ms  
  
--- ping statistics ---  
10 packets transmitted, 10 packets received, 0.0% packet loss  
round-trip min/avg/max/stddev = 2.756/3.918/7.730/1.907 ms
```

Figura 6: Saída do ping entre as máquinas do teste.

7 Comparação C1 × C2

Os dois cenários fazem dez GETs sequenciais de `/index.html` (2294 bytes) com os scripts `measure-c1.sh` e `measure-c2.sh`. Em C1 cada requisição leva `Connection: close` e abre uma conexão nova; em C2 as dez requisições usam a mesma conexão. As capturas são `capturas/c1.pcapng` e `capturas/c2.pcapng`, feitas no servidor. Os bytes são a soma do tamanho dos quadros (`frame.len`) nos dois sentidos, e o tempo vai do primeiro SYN ao último ACK do encerramento.

Métrica	C1	C2	Economia
Handshakes TCP completos	10	1	—
Total de pacotes	120	67	44,2 %
Bytes totais	32 630	29 070	10,9 %
Tempo total (ms)	105,6	64,5	39,0 %

Tabela 6: C1 (conexão nova por requisição) e C2 (conexão persistente). Economia = $100 \times (C1 - C2)/C1$.

A economia de bytes é bem menor que a de pacotes porque os pacotes poupados são de controle, com 54 a 66 bytes cada, enquanto o grosso dos bytes é o conteúdo, igual nos dois cenários (dez vezes o mesmo arquivo). Não houve retransmissão nem RST em nenhuma das capturas.

8 Overhead de abertura e encerramento em C1

As dez conexões de C1 são idênticas: 12 pacotes e 3263 bytes cada. Sete desses pacotes não carregam dado nenhum e servem só para abrir e fechar a conexão.

Etapa	Por conexão	Pacotes	Bytes
Abertura (SYN, SYN/ACK, ACK)	3 pacotes, 192 bytes	30	1920
Encerramento (FIN, ACK, FIN, ACK)	4 pacotes, 228 bytes	40	2280
Total	7 pacotes, 420 bytes	70	4200

Tabela 7: Pacotes e bytes de controle de conexão em C1. As duas últimas colunas somam as dez conexões.

Abrir e fechar conexões consome 70 dos 120 pacotes de C1 (58,3%) e 4200 dos 32 630 bytes (12,9%). Uma ressalva na contagem: o ACK do cliente no encerramento (quadro 10 da Tabela 3) confirma ao mesmo tempo os últimos dados e o FIN do servidor. Contamos esse pacote como encerramento. Em C2 o cliente confirma cada resposta com um ACK separado, e isso explica a diferença entre os cenários:

- Pacotes: $9 \times 7 = 63$ de controle a menos, e 10 ACKs de dados a mais em C2. $63 - 10 = 53 = 120 - 67$.
- Bytes: $9 \times 420 = 3780$ de controle a menos, mais 380 bytes do cabeçalho `Connection: close` (19 bytes na requisição e 19 na resposta, dez vezes), menos 600 bytes dos 10 ACKs. $3780 + 380 - 600 = 3560 = 32\,630 - 29\,070$.

9 Análise em função do RTT

Cada conexão nova custa um RTT antes que a requisição possa sair: o cliente envia o SYN, espera o SYN/ACK e só então manda o ACK e o GET. Na Tabela 3 isso aparece entre os quadros 2 e 3. A troca requisição/resposta custa mais um RTT, e esse custo existe nos dois cenários. C1 faz dez handshakes e C2 faz um, então C1 gasta nove RTTs a mais.

O encerramento não acrescenta espera. Na captura, o cliente manda o FIN de uma conexão e o SYN da seguinte no mesmo instante (quadros 11 e 12, com $2\,\mu\text{s}$ de diferença), sem aguardar o último ACK.

Diferença medida, $t_{C1} - t_{C2}$	41,2 ms
Previsão com o RTT do ping, $9 \times 3,918\text{ ms}$	35,3 ms
RTT médio dos dez handshakes de C1 (SYN/ACK até o ACK)	4,484 ms
Previsão com o RTT dos handshakes, $9 \times 4,484\text{ ms}$	40,4 ms

Tabela 8: Diferença de tempo entre os cenários e previsão do modelo.

A diferença medida equivale a 10,5 RTTs do ping, contra os 9 previstos. O motivo é que o RTT durante o teste foi maior que o do ping: medido na própria captura, pelo intervalo entre o SYN/ACK do servidor e o ACK do cliente, ele ficou entre 3,4 e 5,5 ms, com média de 4,484 ms. Com esse valor, nove handshakes dão 40,4 ms, a 0,8 ms do medido.

Essa concordância tem uma coincidência por trás. Olhando o intervalo entre GETs consecutivos, C1 leva em média 10,66 ms por requisição e C2 leva 4,62 ms (da segunda requisição em diante). A diferença, 6,0 ms, é um RTT de handshake mais cerca de 1,5 ms, que vêm do tempo de resposta do servidor (cerca de 1,0 ms em C1 e 0,6 ms em C2, porque cada conexão nova passa pelo `accept` e pela fila do *pool*) e do trabalho do cliente para criar outro socket. Em sentido contrário, a primeira requisição de C2 foi lenta: 14,0 ms, com 6,5 ms só para o servidor começar a responder, contra cerca de 0,6 ms nas seguintes. A captura não mostra a causa; o servidor estava parado havia alguns minutos. Os dois efeitos quase se cancelam no total.

10 Conclusão

Com dez requisições e RTT em torno de 4 ms, a conexão persistente usou 44 % menos pacotes, 11 % menos bytes e terminou 41 ms antes. Quase todo esse tempo vem dos nove handshakes que C2 não faz.

O ganho da persistência depende de dois fatores: quantas conexões ela evita e quanto custa cada uma. O custo de cada conexão evitada é de um RTT, então o tempo poupado é aproximadamente $(N - 1) \times \text{RTT}$ para N requisições. Na nossa rede local isso deu 41 ms. Em um enlace com RTT de 100 ms, como um acesso intercontinental ou uma rede móvel congestionada, os mesmos nove handshakes custariam cerca de 900 ms, enquanto o tempo de transferir os 2294 bytes mudaria pouco. Uma página com dezenas de recursos pequenos aumenta N e tem o mesmo efeito. O ganho seria maior que o medido, portanto, em redes de RTT alto e com muitas requisições curtas para o mesmo servidor. Com HTTPS a diferença cresceria ainda mais, porque cada conexão nova também paga o handshake do TLS.